Laboratorium 03 - Dart – Funkcje, Klasy, Listy i Testy

1. Funkcje

Funkcja to blok kodu, który można wielokrotnie wywoływać.

Podstawowy przykład:

```
int add(int a, int b) {
  return a + b;
}

void main() {
  print(add(3, 5)); // 8
}
```

Skrócona składnia arrow:

```
int multiply(int a, int b) => a * b;
```

Parametry nazwane i wartości domyślne

```
String greet({required String name, String punctuation = "!"}) {
  return "Hello, $name$punctuation";
}

void main() {
  print(greet(name: "Anna"));           // Hello, Anna!
  print(greet(name: "Marek", punctuation: " :)"));
}
```

---

2. Klasy i obiekty

Klasa opisuje dane i zachowania obiektu.

```
class Car {
  final String brand;
  final String model;
  int mileage;

  Car(this.brand, this.model, {this.mileage = 0});

  void drive(int kilometers) {
    mileage += kilometers;
  }

  String info() => "$brand $model with $mileage km";
}

void main() {
  final myCar = Car("Toyota", "Corolla");
  myCar.drive(120);
  print(myCar.info()); // Toyota Corolla with 120 km
}
```

Gettery i settery oraz toString:

```
class Rectangle {
  double width;
  double height;

  Rectangle(this.width, this.height);

  double get area => width * height;

  set renameWidth(double value) {
    if (value <= 0) throw ArgumentError("Width must be positive");
    width = value;
  }

  @override
  String toString() => "Rectangle $width x $height area $area";
}
```

---

3. Listy

Lista przechowuje wiele elementów. Typ elementów zapisujemy w nawiasach ostrych.

```
void main() {
  List<String> fruits = ["Apple", "Banana", "Orange"];

  fruits.add("Pear");
  print(fruits.first); // Apple
  print(fruits.length); // 4

  print('---fruits---');
  for (final fruit in fruits) {
    print(fruit);
  }
  print('---fruits---');

  final upper = fruits.map((f) => f.toUpperCase()).toList();
  final longNames = fruits.where((f) => f.length > 5).toList();

  // Spread i kolekcje warunkowe
  final moreFruits = ["Kiwi", ...fruits];
  final bool addMango = true;
  final fancy = [
    "Grape",
    if (addMango) "Mango",
    for (final f in fruits) "Fresh $f",
  ];

  print('upper method -> $upper');
  print('longNames method -> $longNames');
  print('moreFruits method -> $moreFruits');
  print('fancy method -> $fancy');
}
```

---

4. Proste testy jednostkowe

Do testów używamy pakietu `test`.

pubspec.yaml

```
dev_dependencies:
```

```
  test: ^1.25.0
```

Funkcje do przetestowania np. w lib/math_utils.dart

```dart
int add(int a, int b) => a + b;

String reverse(String input) => input.split("").reversed.join();
```

Plik testu test/math_utils_test.dart

```dart
import 'package:test/test.dart';
import '../lib/math_utils.dart';

void main() {
  group("math utils", () {
    test("add adds two numbers", () {
      expect(add(2, 3), equals(5));
      expect(add(-1, 1), equals(0));
    });

    test("reverse reverses a string", () {
      expect(reverse("Dart"), equals("traD"));
      expect(reverse("anna"), equals("anna"));
    });
  });
}
```

Uruchom w terminalu

```
dart test
```

Zadania praktyczne 💻

- **Zadanie 1: Funkcja i parametry nazwane**
  Napisz funkcję `formatName` z parametrami nazwanymi `firstName` i `lastName` oraz opcjonalnym `uppercase` domyślnie `false`. Gdy `uppercase` jest `true` zwróć wynik wielkimi literami.
- **Zadanie 2: Klasa z metodami**

Utwórz klasę `BankAccount` z polami `owner` i `balance`. Dodaj metody `deposit` oraz `withdraw` z prostą walidacją. Dodaj metodę `summary` zwracającą czytelny opis stanu konta.

- **Zadanie 3: Operacje na liście**
  Mając listę `List<int> numbers`, zwróć nową listę z kwadratami liczb parzystych posortowaną rosnąco. Użyj `where`, `map`, `toList` oraz `sort`.
- **Zadanie 4: Test jednostkowy**
  Napisz funkcję `isAdult` która przyjmuje `age` i zwraca `true` gdy `age >= 18`. Dodaj testy obejmujące wartości graniczne.

Przydatne linki 🔗

- Dokumentacja funkcji: https://dart.dev/language#functions
- Dokumentacja klas: https://dart.dev/language#classes
- Pakiet test: https://pub.dev/packages/test